

Module 8 — Stretch: Query Router

***Honors Track.** This stretch is for learners who have completed all required Week 8 work (Reading, Drill, Lab, Integration), are On Track or Advanced, and are attending consistently. See the program's Honors Track policy for full eligibility. Stretch participation is graded but not required for program completion; it counts toward Honors distinction.*

In this stretch you will build a routing layer that classifies an incoming query into one of three types — **factoid**, **semantic**, or **mixed** — and dispatches it to the lab's BM25, dense, or hybrid retriever accordingly. You will then evaluate your routed retriever against the lab's three baselines on the 60-pair labeled set.

The pedagogical goal is to move from *picking one retriever* (lab) to *picking the right retriever per query* — and to confront the cost: when does routing beat the best single retriever, and when does it lose?

This stretch is also the M8 contribution to the **multi-agent capstone scaffold**. Learners who complete all three multi-agent stretches (M8 routing, M10 multi-service orchestration, M11 distributed observability) end the course with the component skills to build a multi-agent system at capstone time.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: None. The lab's `evaluate_retriever` and the three baseline retrievers are provided as starter helpers in the stretch repo.

What You Will Produce

This stretch is a **standalone repo** spawned from the GitHub Classroom assignment `M8-S8-query-router`. New files (or completed implementations) inside the spawned repo:

- `router.py`** — your `classify_query(query)` and `routed_search(client, query, k, embedder)` implementations.
- `routing_report.md`** — 300–500-word writeup with three required sections.
- PR description** — per-query-type classifier accuracy, `evaluate_retriever` output for the routed retriever, and comparison to the lab's three baselines (BM25 / dense / hybrid).

Setup

***Submission flow** — all Module 8 assignments. Module 8 uses a fork-and-submit flow instead of GitHub Classroom. Same pattern as the drill, lab, and integration.*

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m8-s8 and click **Fork** → owner = your personal GitHub account → **Create fork**.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab → **I understand my workflows, go ahead and enable them**.
- Clone your fork (not the upstream repo), then branch:**

```
git clone https://github.com/<your-github-username>/m8-s8.git
cd m8-s8
git checkout -b stretch-query-router
```

- Install dependencies (cached from the lab — should be fast):

```
pip install -r requirements.txt
```

- Bring up Weaviate (same image as the lab):

```
docker run -d --name weaviate-stretch \
  -p 8080:8080 \
  -e DEFAULT_VECTORIZER_MODULE=none \
  -e ENABLE_MODULES= \
  -e PERSISTENCE_DATA_PATH=/var/lib/weaviate \
  semitechnologies/weaviate:1.24.10
```

- Ingest the corpus once with the provided helper:

```
python ingest.py
```

- Instantiate the embedder** that `routed_search(...)` will receive. Add this at module scope in `router.py` (mirroring the lab's pattern):

```
from sentence_transformers import SentenceTransformer
embedder = SentenceTransformer("all-MiniLM-L6-v2")
```

The autograder constructs the same embedder and passes it as the fourth positional argument to `routed_search(client, query, k, embedder)` — without instantiating it yourself, calling `routed_search` from the REPL or your own test scaffolding will fail with a missing-argument error.

Tasks

- Classify the query — `classify_query(query: str) -> str`

Two valid implementations are accepted:

- Rule-based:** heuristics over query length, presence of named entities (regex for capitalized multi-word phrases), presence of digits, exact-phrase quotes. Return one of `"factoid" | "semantic" | "mixed"`.
- Embedding-similarity-based:** maintain three labeled exemplar query sets (10 each); embed the incoming query; classify to the nearest exemplar centroid in embedding space.

Pick one and explain your choice in `routing_report.md`.

- Build the router — `routed_search(client, query, k, embedder) -> list[str]`

Calls `classify_query`; dispatches to `bm25_search` / `dense_search` / `hybrid_search` (provided in `retrieval_helpers.py`) accordingly. Returns ordered `list[str]` of `doc_id`s.

- Evaluate

Run the provided `evaluate_retriever` against your routed retriever on the 60-pair labeled set in `data/retrieval_eval.jsonl`. Compare to pure BM25, pure dense, and pure hybrid (the lab's three baselines).

- Report — `routing_report.md` (300–500 words)

Three required sections:

- Per-query-type accuracy of the classifier.** If you used a rule-based classifier, label a held-out subset (~20 queries) by hand and report accuracy. If you used embedding-based, the held-out set is automatic from your exemplar split.
- End-to-end recall@5 and MRR of the router**, with a comparison table against the three baselines.
- When does routing beat the best single retriever, when does it lose, why?** Cite specific queries from the labeled set.

Submission

- Commit `router.py`, `routing_report.md`, and any supporting files.
- Push the branch (`git push origin stretch-query-router`) and open a **PR within your fork** — base branch `main`, compare branch `stretch-query-router`, **base repository = your fork** (change the dropdown from `LevelUp-Applied-AI/m8-s8` to `<your-username>/m8-s8`). If Actions are disabled on your fork, see Setup step 2 — re-enable, then `git commit --allow-empty -m "ci: trigger" && git push` to re-run CI.
- PR description must include:
 - Per-query-type classifier accuracy.
 - `evaluate_retriever` output for the routed retriever.
 - Comparison to the lab's three baselines.
- Paste your PR URL into TalentLMS → Module 8 → Tuesday Stretch to submit this assignment.

Grading

This stretch is **rubric-graded by a TA** across 3 weighted dimensions (Classifier correctness 40%, Routed retriever metrics 30%, Writeup analysis quality 30%), scored Exemplary (4) / Proficient (3) / Developing (2) / Insufficient (1). The weighted sum is scaled to 100.

The autograder verifies output shape and a behavioral floor on a fixture subset. The TA rubric drives the final score.

Multi-Agent Capstone Hook

The routing pattern you implement here is the same pattern you would use to dispatch to an LLM agent vs. a deterministic tool vs. a search index. M10 extends this to multi-service orchestration; M11 adds per-service decision logging. If you do all three, your capstone has the option of a multi-agent system as a built-in scaffolded path.